

Third-Party Political Advertising on Facebook Around the 2022 Australian Federal Election

DATA7201 Data Analytics at Scale — Project Report

Student Number: s3348393

May 16, 2026

Abstract

This report analyses four years of Australian political advertising from the Facebook Ad Library (March 2020 to February 2024), with a focus on third-party advertisers around the May 2022 federal election. Approximately 320 GB of JSON across 4,500 daily snapshots were ingested with PySpark on YARN, deduplicated, and windowed to six months either side of election day, which reduced the corpus from 40 million to 5.7 million ads. A pandas UDF tagged candidate, party, and government advertising via token matching against the Australian Electoral Commission candidate roll. The remaining ads were clustered with Latent Dirichlet Allocation and hand-labelled, and commercial clusters were removed, leaving about 95,000 advocacy ads. The corpus was then analysed in two ways: a deterministic temporal classifier based on persistence and centroid metrics, and a hashtag-supervised logistic-regression text classifier. Third-party advertising splits into distinct temporal patterns, with election-only advertisers dominating by count and Greenpeace Australia dominating the ongoing tier by spend. The content classifier identifies advertisers whose messaging pivoted to electoral themes around polling day even when their spend pattern did not, with the Australian Christian Lobby showing the largest content shift. Combining temporal and content methods is needed to classify third-party political advertising. Neither metric alone is enough.

Contents

1	Introduction	3
2	Dataset Analytics	3
2.1	Data ingestion and preparation	3
2.2	Identifying government advertising	4
2.3	Removing commercial noise via LDA	5
2.4	Temporal analysis: persistence and centroid bands	5
2.5	Content analysis: hashtag-supervised classifier	6
3	Discussion and Conclusions	7
A	Code	8

1 Introduction

Winning elections requires reaching voters, and there are many ways to do this. Online advertising is attractive because it allows a message to reach a wide audience with minimal effort. Designing an ad, placing it online, and paying the bill is easier than traditional methods such as door-knocking or physical signage.

Political advertising is not limited to political parties. Third-party groups also participate in election campaigns, and studying digital advertising allows us to examine who these groups are, what they spend, and when they choose to spend it. The Facebook Ad Library API publishes every sponsored political post targeted at Australian users from March 2020 to February 2024, covering three major political events: the COVID-19 pandemic response, the May 2022 federal election, and the October 2023 Voice to Parliament referendum. This report focuses on the 2022 federal election.

At approximately 320 GB of JSON files, the dataset cannot be analysed on a conventional workstation. A single machine cannot hold it in memory, and the joins required for deduplication and classification would be prohibitively slow. Distributed analytics frameworks are the natural fit. PySpark is well suited to this task because it distributes both storage and computation across a cluster, loads data lazily, and its DataFrame API supports efficient analysis across thousands of files. We also leverage Spark MLlib for both the unsupervised topic modelling (LDA) and the supervised text classifier (logistic regression with TF-IDF features).

The investigation was split into three steps: data ingestion and preparation, exploratory analysis, and election-window analysis. These are discussed in the following sections.

2 Dataset Analytics

2.1 Data ingestion and preparation

The data consists of approximately 4,500 JSON files containing around 40 million rows spanning February 2020 to March 2024. Each file is a snapshot of all Australian political advertisements active at that moment, so the same ad appears in multiple files. Each row represents a single ad with around 25 columns covering the ad purchaser, Facebook page, creative text, spend, and impressions. The schema varies over time, with a notable change in June 2022 and another in August 2023; data becomes sparse after August 2023.

A null-fraction analysis was performed to visualise the schema drift. Automatic schema inference collected the superset of all fields across the dataset, and the per-column null fraction by month was plotted as a heatmap (Figure 1). Columns were then modified and merged to produce a consistent schema across the full date range. The numeric fields were stored as nested structs of string-typed lower and upper bounds; these were cast to numeric types and a midpoint was computed for each field, allowing downstream aggregations to work on a single value.

Deduplication was achieved by extracting the snapshot date from each filename and using it to sort snapshots chronologically, grouped by ad ID. A `row_number()` window operation tagged each row with an `ad_seq_no` where 1 indicates the most recent snapshot per ad. Filenames used inconsistent date conventions (some 7-digit, some 8-digit) and required special handling. The cleaned data was windowed to six months either side of the May 2022 federal election and written as Parquet to HDFS, reducing the corpus from approximately 40 million to 5.7 million rows.

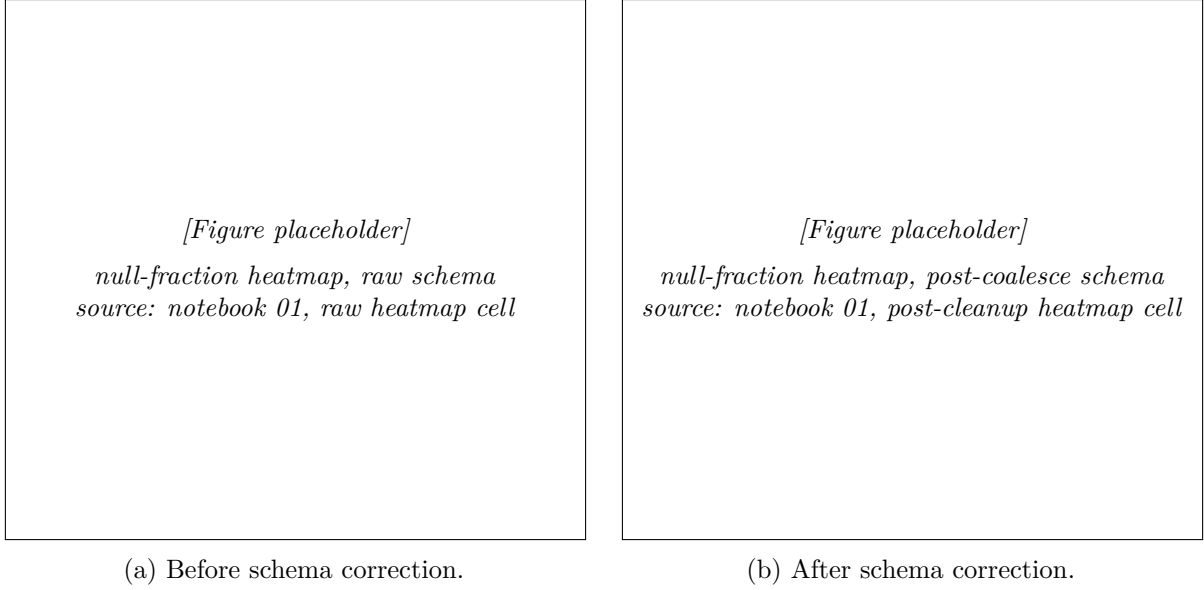


Figure 1: Null-fraction heatmaps (column $\times$ month) before and after the schema coalesce step. The raw view (left) shows the June 2022 singular-to-array transition and the August 2023 breakage clearly. The cleaned view (right) has the duplicate fields merged and a single consistent schema across the window.

2.2 Identifying government advertising

To focus on third-party organisations the government advertising had to be removed. This included political-party advertisements and candidate advertisements. The 2022 House of Representatives and Senate candidate rolls were sourced from the Australian Electoral Commission. A straight join was not practical because page names can be misrepresentative and the byline can be paid by an aide. A pandas UDF was written instead, using Arrow to batch data between the JVM and Python workers for a 3 to 10 times speed-up over a row-by-row UDF. The UDF applied three rules in order with the first match winning, summarised in Table 1.

Rule	Logic and output
Candidate name	The candidate’s tokenised name (surname plus first given name) is a subset of either the page-name tokens or the byline tokens. Tagged as candidate with party from the AEC roll.
Party byline signature	A tokenised party signature from <code>aec_parties.csv</code> is a subset of the byline tokens, e.g. {liberal, national} $\rightarrow$ LNP. Signatures are tested longest-first so LNP matches before LP. Tagged as party_org .
Government byline signature	The byline tokens contain one of {department}, {australian, government}, or {australian, electoral, commission}. Tagged as government . {commonwealth} was excluded to avoid collisions with Commonwealth Bank.

Table 1: Three rules applied by the byline-matching pandas UDF in order, first match wins.

Every ad ended up tagged with a `match_type` (`candidate`, `party_org`, `government`, or `null`) and a `political_party` code where applicable. The row counts are shown in Table 2. Rows with `match_type = null` were the non-government residual required for the third-party analysis.

<code>match_type</code>	total rows	unique ads
null	4,511,742	105,814
party_org	649,857	23,932
candidate	536,232	20,644
government	98,660	1,188

Table 2: Rows and unique ads after government, party, and candidate classification.

2.3 Removing commercial noise via LDA

The non-government residual still contained commercial noise. A natural language processing pipeline was built to remove it. The pipeline used Latent Dirichlet Allocation to cluster the residual ads into topic groups based on the ad text body. LDA was selected because it is unsupervised. There were no labelled examples to train on, and LDA scales well on Spark MLlib. Text was lowercased, regex-tokenised, stop-words removed, and vectorised with a `CountVectorizer` (`minDF=100`, `maxDF=0.3`, `vocabSize=5,000`). After tuning the number of topics on a sample, $k = 20$ gave the cleanest separation. The resulting topics were hand-labelled with a fine label (e.g. `anti_woodside_petition`) and a coarse category (`climate`, `humanitarian_rights`, `political_advocacy`, `cost_of_living`, or `noise`). Topics labelled `noise` were dropped. The result was a corpus of about 95,000 advocacy ads, written partitioned by category for selective downstream reads.

2.4 Temporal analysis: persistence and centroid bands

The temporal analysis characterised how each advertiser’s spending related to the election. Advertisers were ranked by pre-election spend and the top 20 retained for detailed inspection. Spending was aggregated into a weekly time series per advertiser using `date_trunc('week', ad_creation_date)`.

The raw weekly series was bursty. Most advertisers buy media in short pulses, so any summary metric on raw weekly spend was leveraged by single-week spikes. A four-week centred rolling mean was applied per advertiser. A hard smoothing boundary was placed at election day, with the pre and post halves smoothed independently. This prevented a final pre-election spike from bleeding forward into post-election weeks and softening the cliff edge of advertisers that stopped at the election. The raw series was kept for plotting. Only the summary metrics used the smoothed series.

Two metrics were computed per advertiser. Persistence was the ratio of post-election spend to pre-election spend. Centroid was the weighted-mean week from election day, where negative values indicate spending before the election and positive values indicate spending after.

Two metrics were needed because persistence alone produced false positives. A campaign that trickled spend past election day, or one that advertised on unrelated topics after, both lifted persistence without indicating non-election alignment. Combining persistence with the centroid distinguished these cases.

Advertisers were split into four bands as shown in Table 3. The bands are also plotted on a quadrant in Figure 2.

Band	Rule
election_only	persistence < 0.30 and centroid ≤ 16 weeks
transitional	0.30 ≤ persistence ≤ 0.80
ongoing	persistence > 0.80
off_campaign	persistence < 0.30 and centroid > 16 weeks

Table 3: Four-band classifier applied to top-20 advertisers.

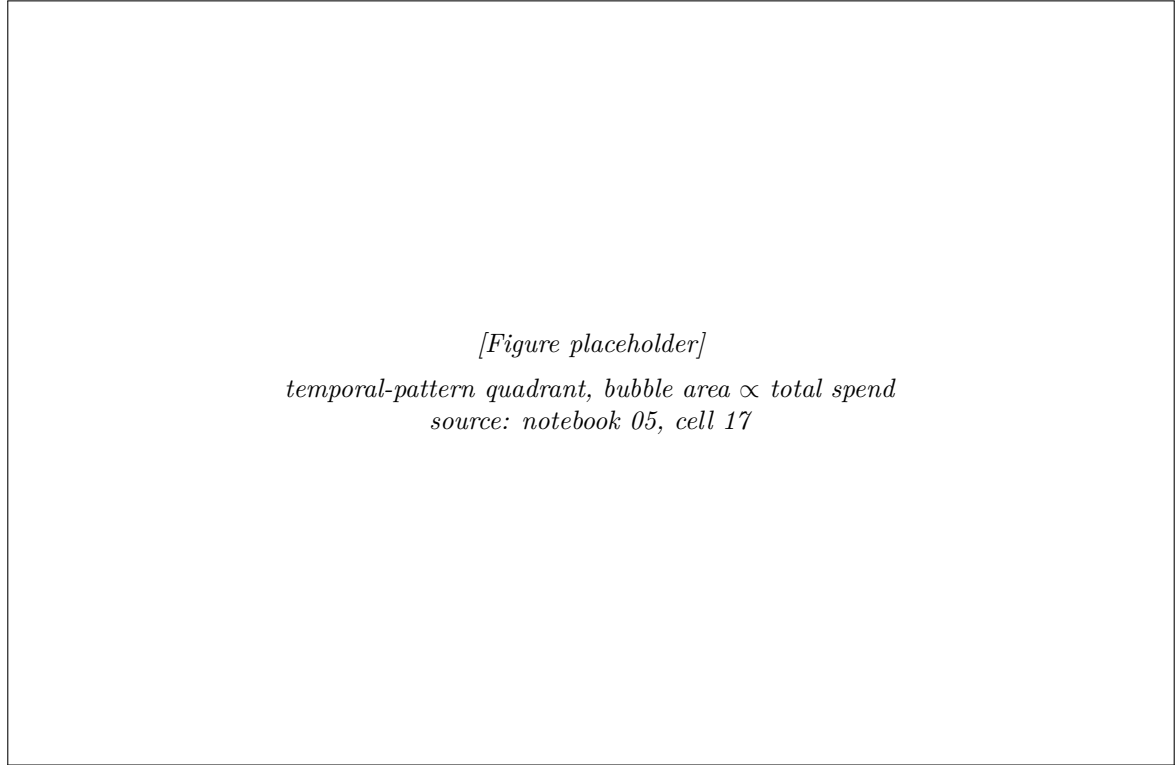


Figure 2: Temporal-pattern quadrant: spend imbalance (x) × centroid timing (y), bubble area ∝ total spend, colour by band. Each quadrant has a clear behavioural interpretation.

2.5 Content analysis: hashtag-supervised classifier

The temporal classifier cannot detect advertisers whose messaging pivoted to electoral themes while their spend remained steady. A content-based classifier was added to capture this. All hashtags were extracted from the residual corpus with a regex and counted by band. Two reference sets were hand-curated from the counts: about 210 political hashtags such as `#federalection` and `#morrissonmustgo`, and about 286 non-political hashtags such as `#breakfreefromplastic` and `#architecture`. Each ad’s text was scanned for matches against both sets. Ads with exactly one match-type set were used as training labels:

- Positive: at least one political hashtag and no non-political hashtag.
- Negative: at least one non-political hashtag and no political hashtag.
- Ambiguous or unlabelled ads were excluded from training but scored at inference.

The hashtags themselves were stripped from the text before vectorisation, so the model had to learn from the surrounding prose rather than the labels. A logistic-regression classifier was

trained over TF-IDF features. Logistic regression is the standard choice for high-dimensional sparse text classification because tree ensembles underperform on sparse input.

The training set contained 2,693 positive ads from 273 bylines and 5,262 negative ads from 306 bylines. Class diversity reduced the risk of advertiser-identity leakage. The held-out test AUC was 0.97 with 91% precision and 87% recall on the positive class. A leave-one-advertiser-out validation across the top 15 contributors gave an ad-weighted recall of 0.66. The lower recall reflects climate-advocacy bylines whose prose overlaps with environmental campaigns in the negative class. Inspection of high-scoring no-hashtag ads showed the model correctly identified political content that the upstream filter had missed, including Family First Party and several state-election candidates.

3 Discussion and Conclusions

The non-party, non-government advocacy ecosystem on Facebook around the 2022 election does not behave as a single block. The temporal classifier identifies four behavioural patterns.

The `election_only` band is the largest by count. It covers roughly half of the top 20 advertisers, including Smart Voting, Solutions for Australia, Advance Australia, the Australian Education Union, GetUp!, It’s Not A Race, Chuffed.org, and Climate 200. These advertisers ramped up before the election and stopped immediately afterwards. Their cumulative spend curves elbow sharply at election day. Keyword analysis confirms that this group is dominated by election-specific language. This is the closest Australian analogue to US PAC-style temporal behaviour.

The `transitional` band contains advertisers that tapered rather than stopping dead. The `ongoing` band contains year-round advocates whose post-election rate matches their pre-election rate. Greenpeace Australia dominates this tier at about \$1.5M of spend across the window. No top-20 advertiser fell into the `off_campaign` band after the upstream commercial filter caught Shell.

The content classifier identifies a different signal. Among advertisers whose spend pattern looked steady, the content classifier reveals that some messaging pivoted toward electoral themes during the campaign period (defined as the four weeks before and after election day). The largest shifts are shown in Table 4.

Advertiser	Outside campaign period	Campaign period	Shift
Australian Christian Lobby	0.51	0.82	+0.31
Thrive By Five	0.42	0.61	+0.19
Amnesty International Australia	0.33	0.52	+0.19
Médecins Sans Frontières	0.29	0.41	+0.12
Australian Conservation Foundation	0.30	0.40	+0.10

Table 4: Top advertisers ranked by content-score shift between the campaign period and the rest of the analysis window. A positive shift indicates a content pivot toward electoral themes.

Greenpeace’s content score dropped during the campaign period from 0.15 to 0.07. Their pre-election spending was focused on the Woodside gas campaign and not on electoral messaging. Climate 200’s content score is high year-round at 0.77, so the campaign-period score of 0.81 shows no pivot because they were already maximally political.

Neither metric alone separates the ecosystem cleanly. Temporal banding catches cliff-edge campaigners and off-cycle outliers but cannot detect content pivots in steady spenders. Content classification catches pivots but cannot speak to spend volume or timing concentration. Together they identify three patterns: PAC-style election-only campaigners visible to both methods,

year-round advocates that pivot messaging during the campaign visible only to the content method, and year-round advocates that stay on issue visible only to the temporal method.

The dataset scale justified PySpark. The 40-million-row deduplication via window functions, the LDA fit on the residual corpus, and the pandas UDF for byline matching all leveraged Spark-specific features. None of these would have fit on a workstation in reasonable time. Hand-curation outperformed automation at two junctures: the LDA topic labels and the hashtag reference sets. Both were short enough for human review but provided strong supervision signal for downstream automation.

A Code

The full analysis pipeline is implemented in six Jupyter notebooks at https://github.com/<USER>/FB_API (replace with the actual URL). Each notebook is self-contained and writes its output as Parquet on HDFS or as CSV in the repository's `data/` directory.

01_data_loading.ipynb Raw JSON ingestion, schema flattening, deduplication, window filtering, write of v1 Parquet.

02_preprocessing.ipynb Candidate, party, and government byline matching via pandas UDF; write of v2 Parquet.

03_topics.ipynb LDA topic modelling, k sweep, topic inspection, output of `topic_terms.csv` for hand-labelling.

04_topic_join.ipynb Join of hand-labelled topics onto v2, filter to political-adjacent residual, write of v3 Parquet.

04b_hashtag_analysis.ipynb Regex extraction of hashtags across v3, count by band, export of `data/hashtags.csv`.

05_election_spenders.ipynb Top-20 ranking, weekly aggregation, election-boundary smoothing, persistence + centroid computation, four-band classification, export of band CSVs and visualisations.

06_election_content_classifier.ipynb Hashtag-supervised logistic-regression text classifier; FP / FN / LOO validation; per-advertiser and per-week scoring; pivot-detection table.

Selected code excerpts follow.

Election-boundary smoothing (nb 05)

```
# Hard smoothing boundary at election day --- pre and post halves
# smoothed independently so cliff-edged advertisers do not bleed forward.
weekly['is_post'] = weekly['week'] >= ELECTION_DATE
weekly['spend_smooth'] = (weekly
    .groupby(['bylines', 'is_post'])['spend']
    .transform(lambda s: s.rolling(window=4, center=True, min_periods=1).mean())
    )
```

Hashtag-supervised labelling (nb 06)


```

pol_re = r'#(' + '|'.join(re.escape(t) for t in political_hashtags) + r')\b'
non_re = r'#(' + '|'.join(re.escape(t) for t in non_political_hashtags) + r')\b'

v3 = v3.withColumn('has_political_hashtag', col('raw_text').rlike(pol_re))
v3 = v3.withColumn('has_non_political_hashtag', col('raw_text').rlike(non_re))

# Strip hashtags before vectorising --- model must learn from surrounding prose.
v3 = v3.withColumn('text', regexp_replace(col('raw_text'), r'#\w+', ' '))

labelled = v3.withColumn('label',
    when( col('has_political_hashtag') & ~col('has_non_political_hashtag'), 1.0)
    .when(~col('has_political_hashtag') & col('has_non_political_hashtag'),
        0.0)
    .otherwise(None))

```